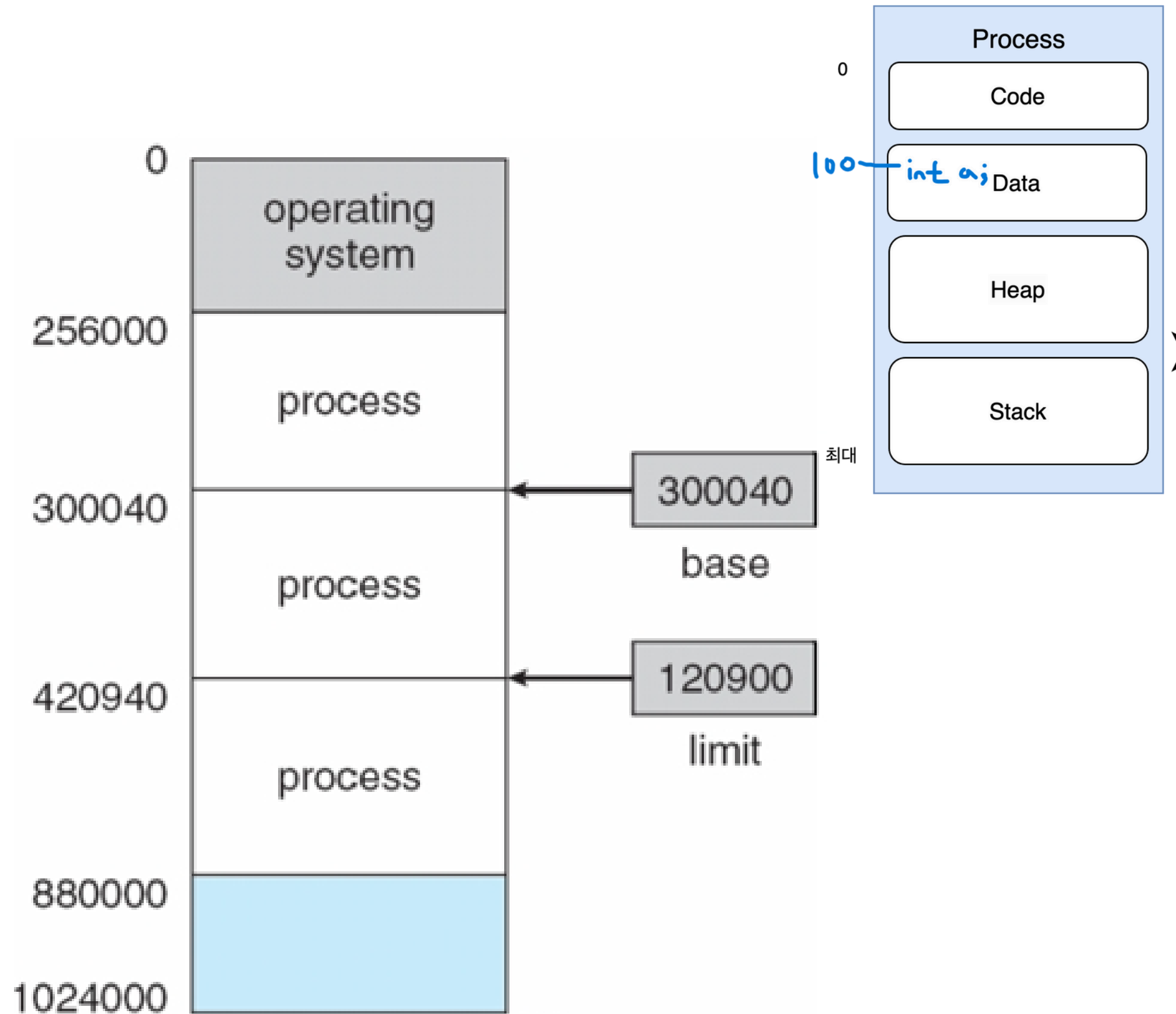


가상메모리: Paging

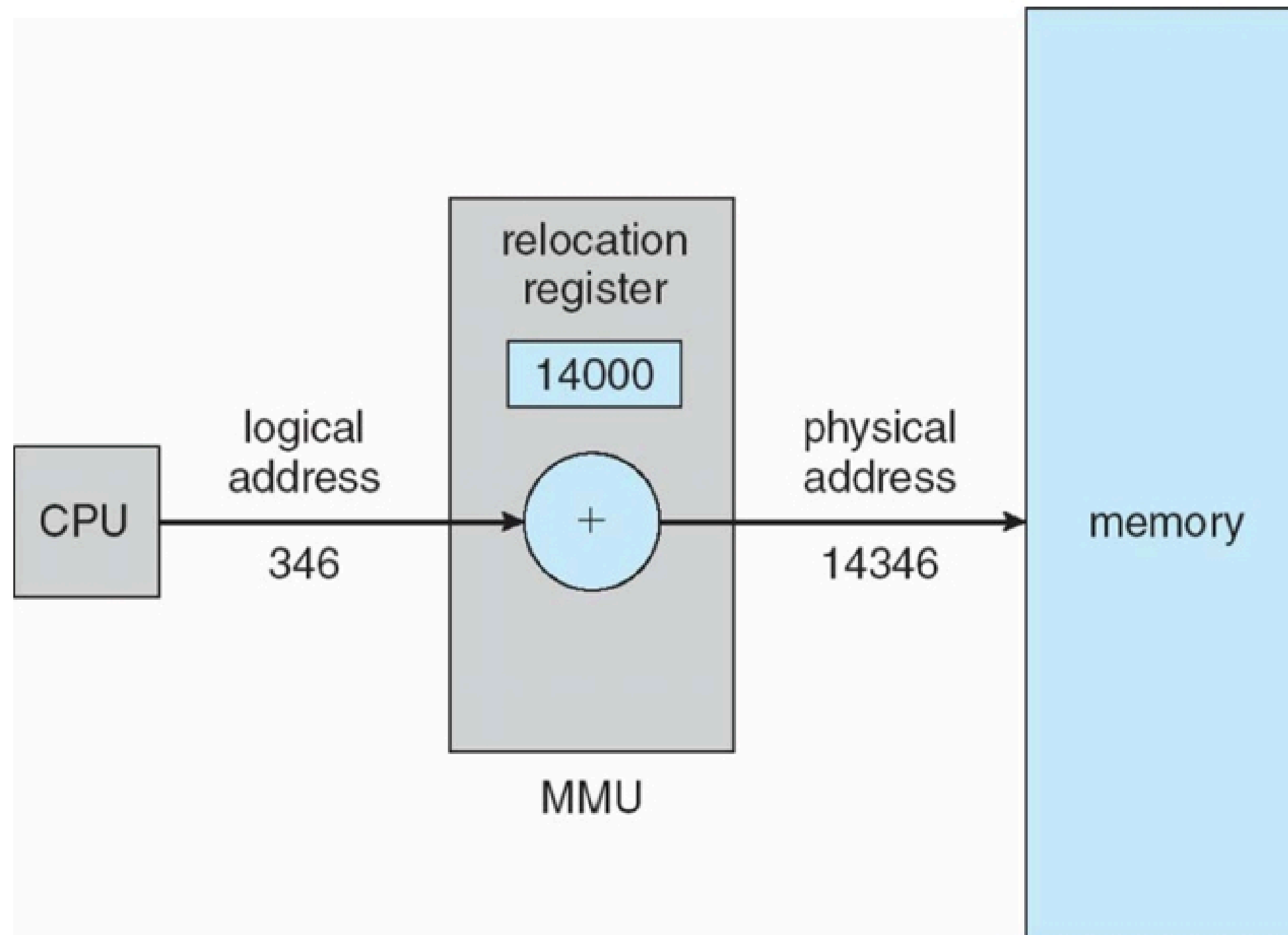
주소 변환

프로그램을 메모리에 적재할 때..



- 데이터는 특정한 주소에서 가져와야한다!
 - 100번지에 a를 저장했다면? cpu는 100번지에서 a를 찾아오면 됨
 - 하지만 우리 컴퓨터의 프로세스는 매우 많음!
 - 여러 프로그램이 동시에 메모리에 올라가있다면?
 - 프로그램마다 주소가 겹치지 않도록 관리해야함
-
- - 운 좋게(?) 메모리 시작지점에 적재되면 좋겠지만...불가능함.
 - 하지만 다른 방법이 있다!

Logical Address to Physical Address

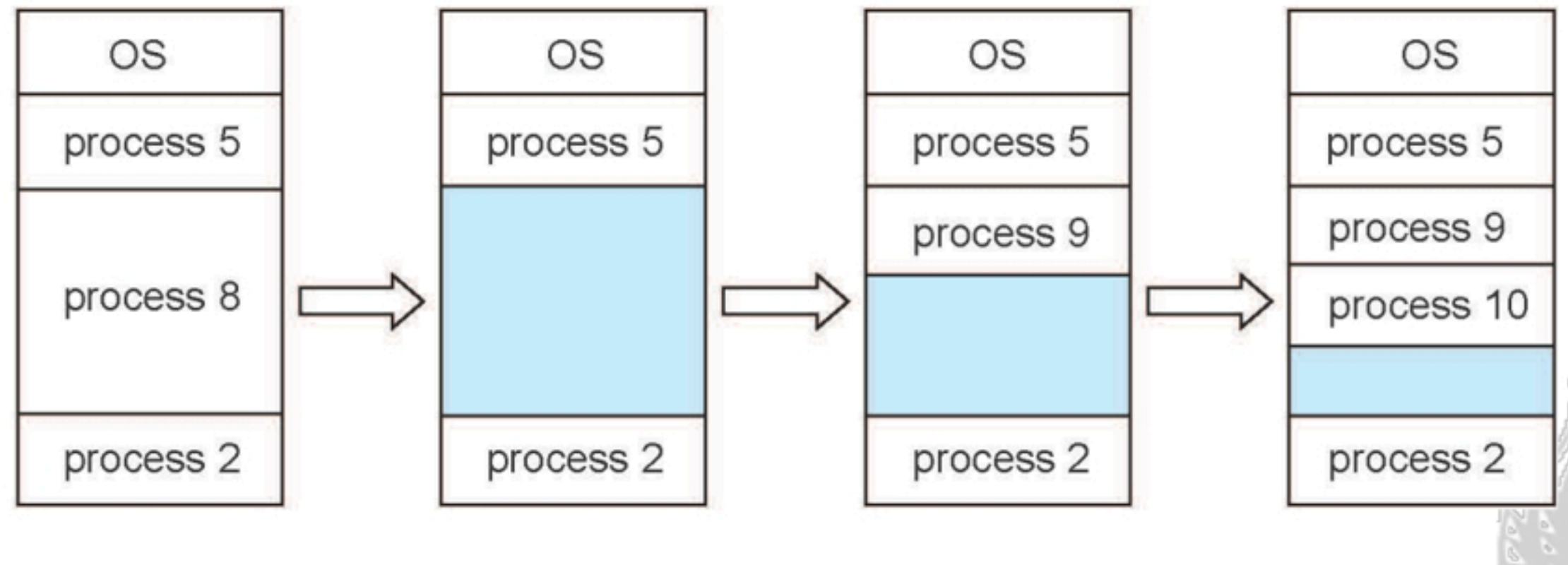


- 주소를 논리와 물리주소로 생각.
 - 논리주소 = 프로세스가 생각하는 주소
 - 물리주소 = 실제 메모리의 주소
- 논리주소를 물리주소로 address translation 하는 특별한 하드웨어 = MMU.
- 346번지(logical)에 a를 저장했다면? MMU가 14000을 더해 14346(physical)에서 a를 찾아오면 됨

Contiguous Allocation

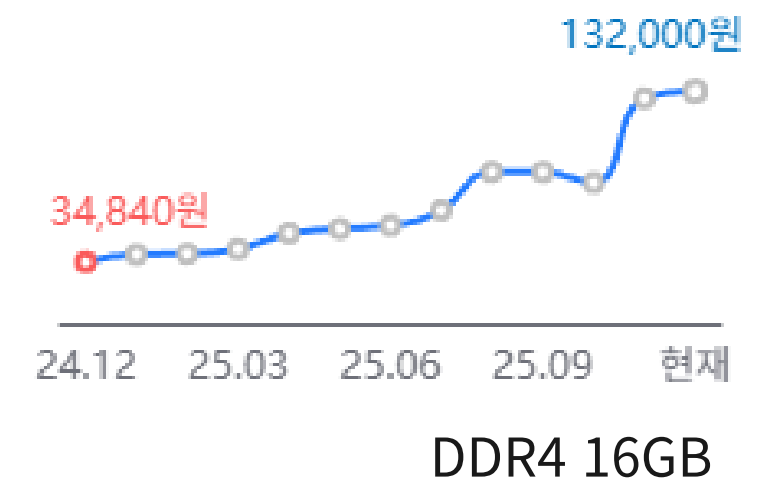
연속 할당

연속 할당

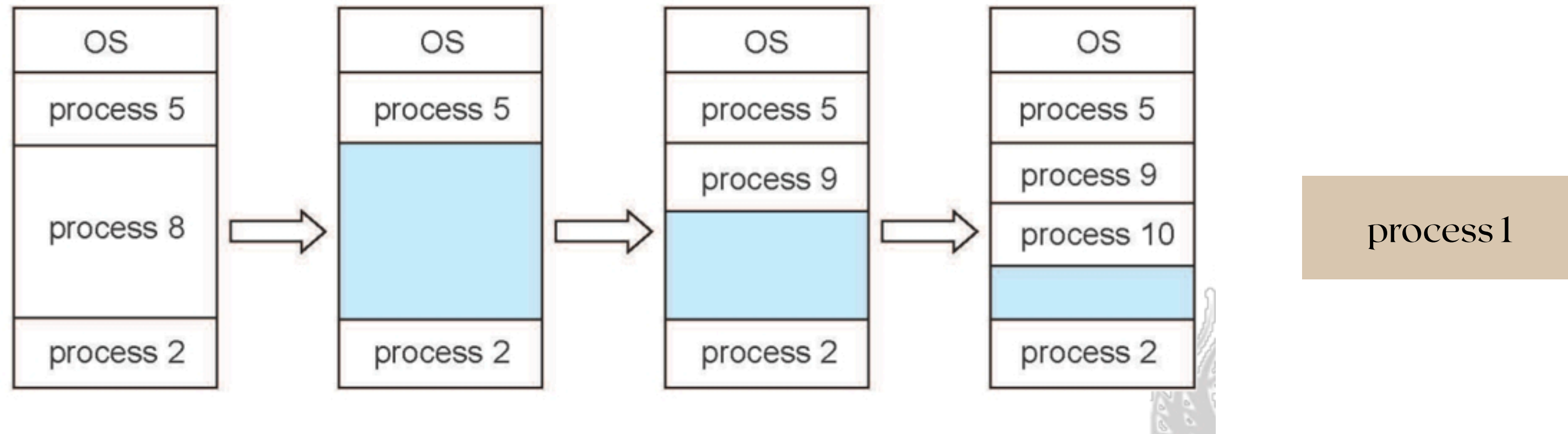


- 프로세스에 순서대로 적재하는 방법
- 문제점?
 - 단편화(Fragmentation)
 - 스와핑(swapping) :특히 context switch

메모리 싸니까 그냥 더 사자?
→ 그 말이 맞는 때가 있었지만..



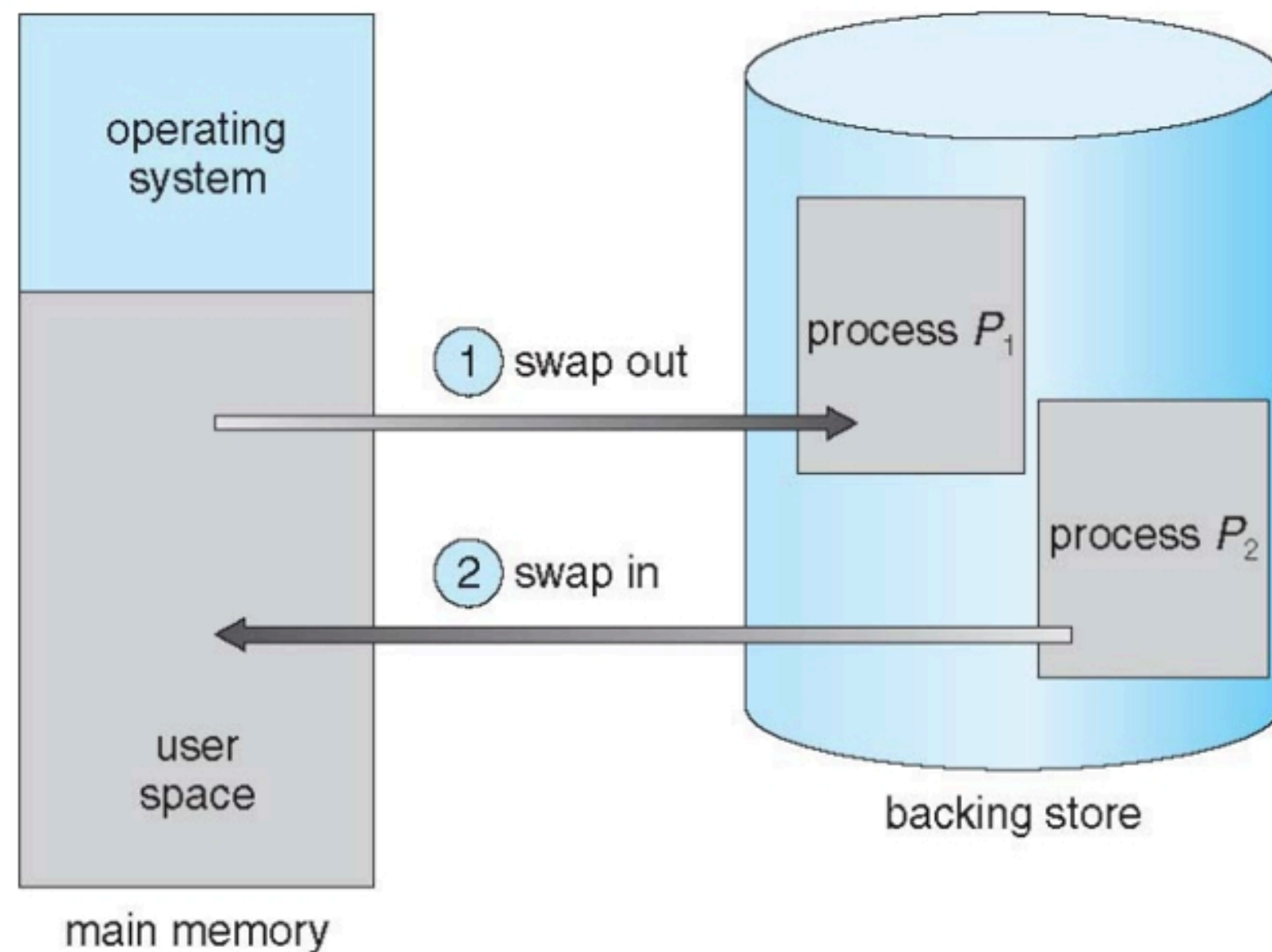
문제점1 : 단편화



- 프로세스 9가 끝나고 process 10이 들어오려면?
- 외부단편화: 낭비되는 hole 공간
- 내부단편화: 할당한 메모리를 프로세스가 전부 안씀
- Compaction?: 배열 중간이 비어있다고 shift불가

문제점2 :스와핑

1. 메모리에 적재된 프로세스를 스토리지로 내보냄.
2. 스토리지에서 사용할 프로세스를 메모리로 적재함.



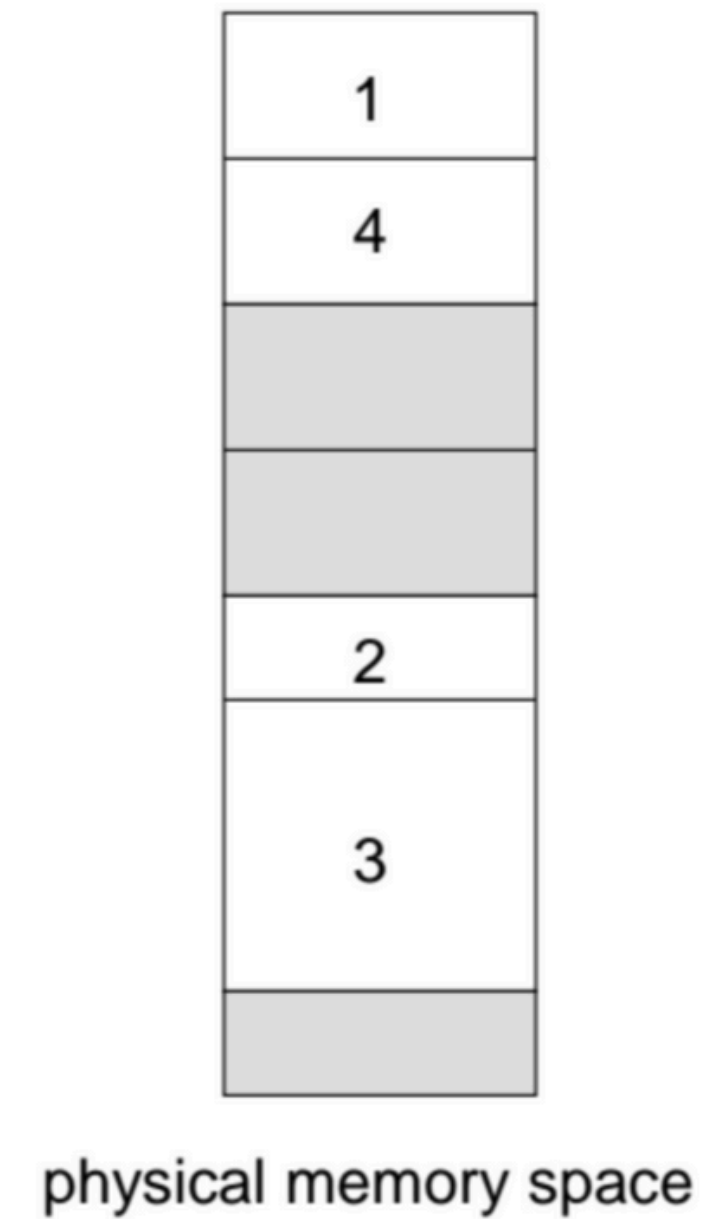
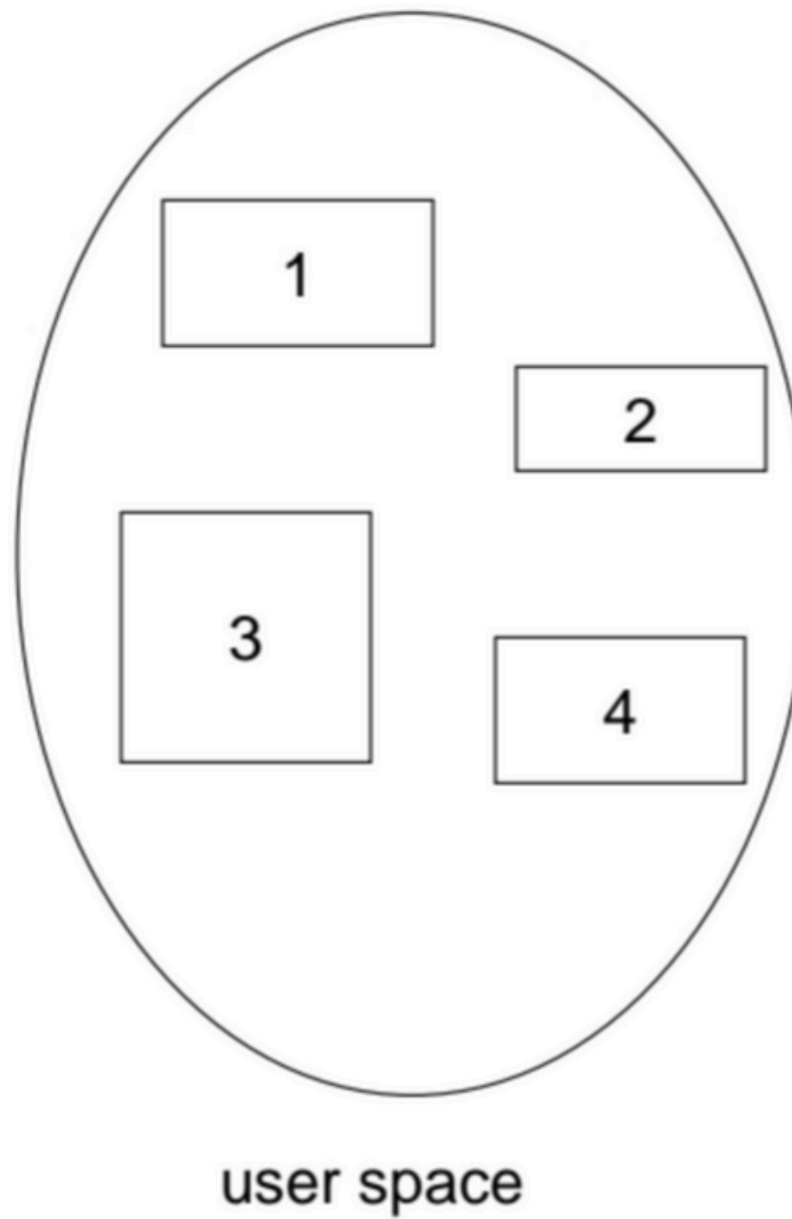
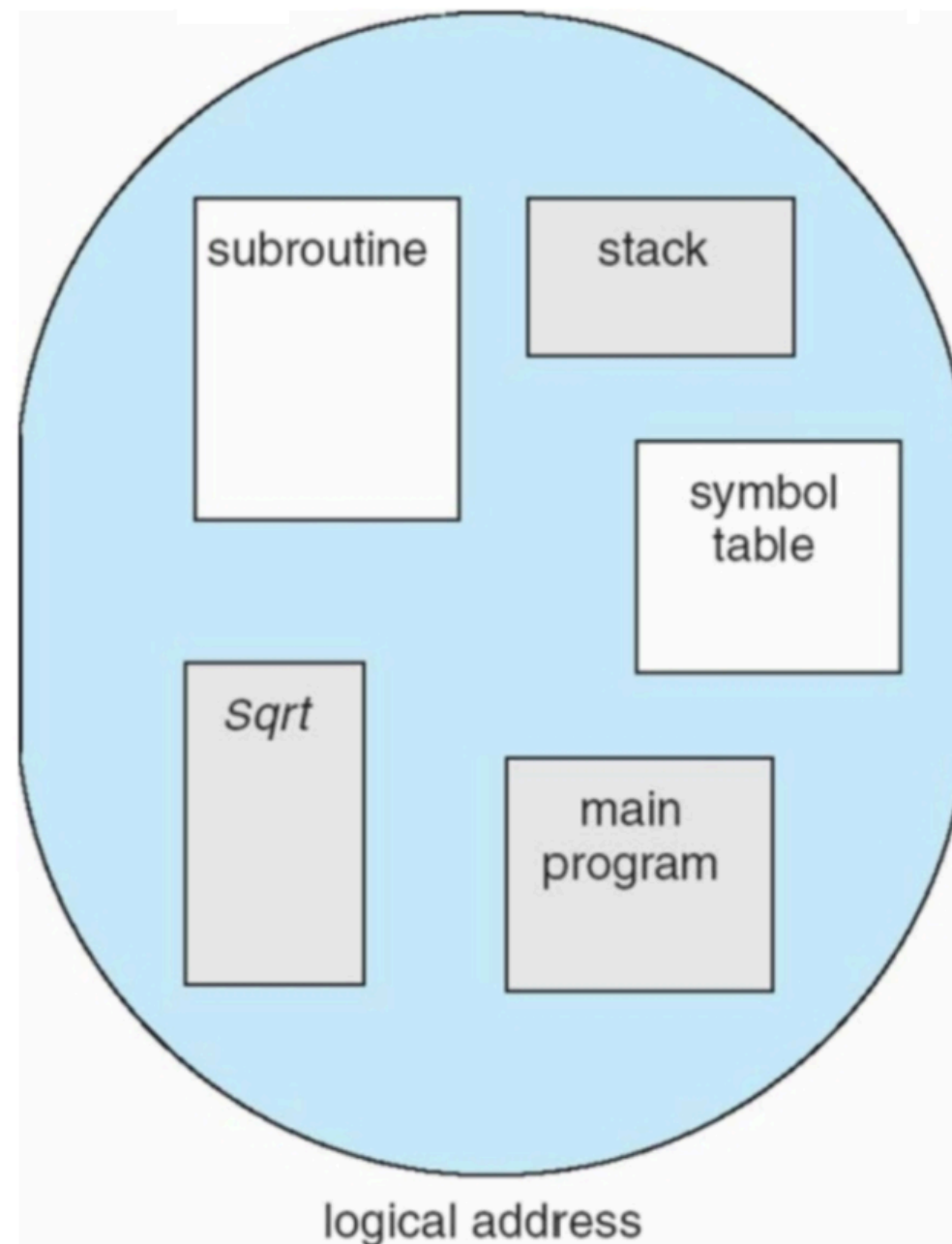
- 컨텍스트 스위칭이 일어나면?

Segmentation

Segmentation

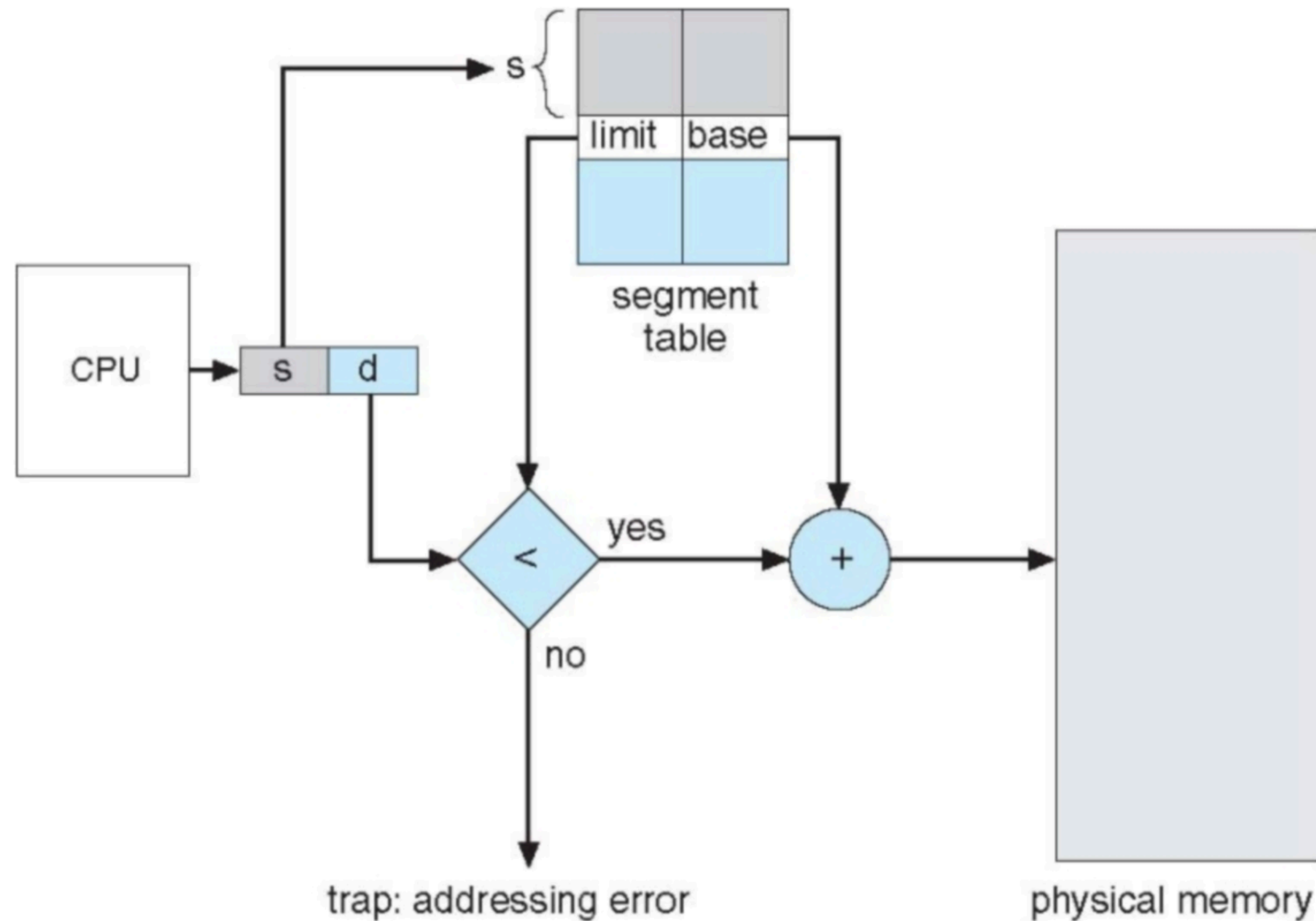
segmentation

프로세스를 논리적 내용(=세그먼트)으로 잘라서 메모리에 배치



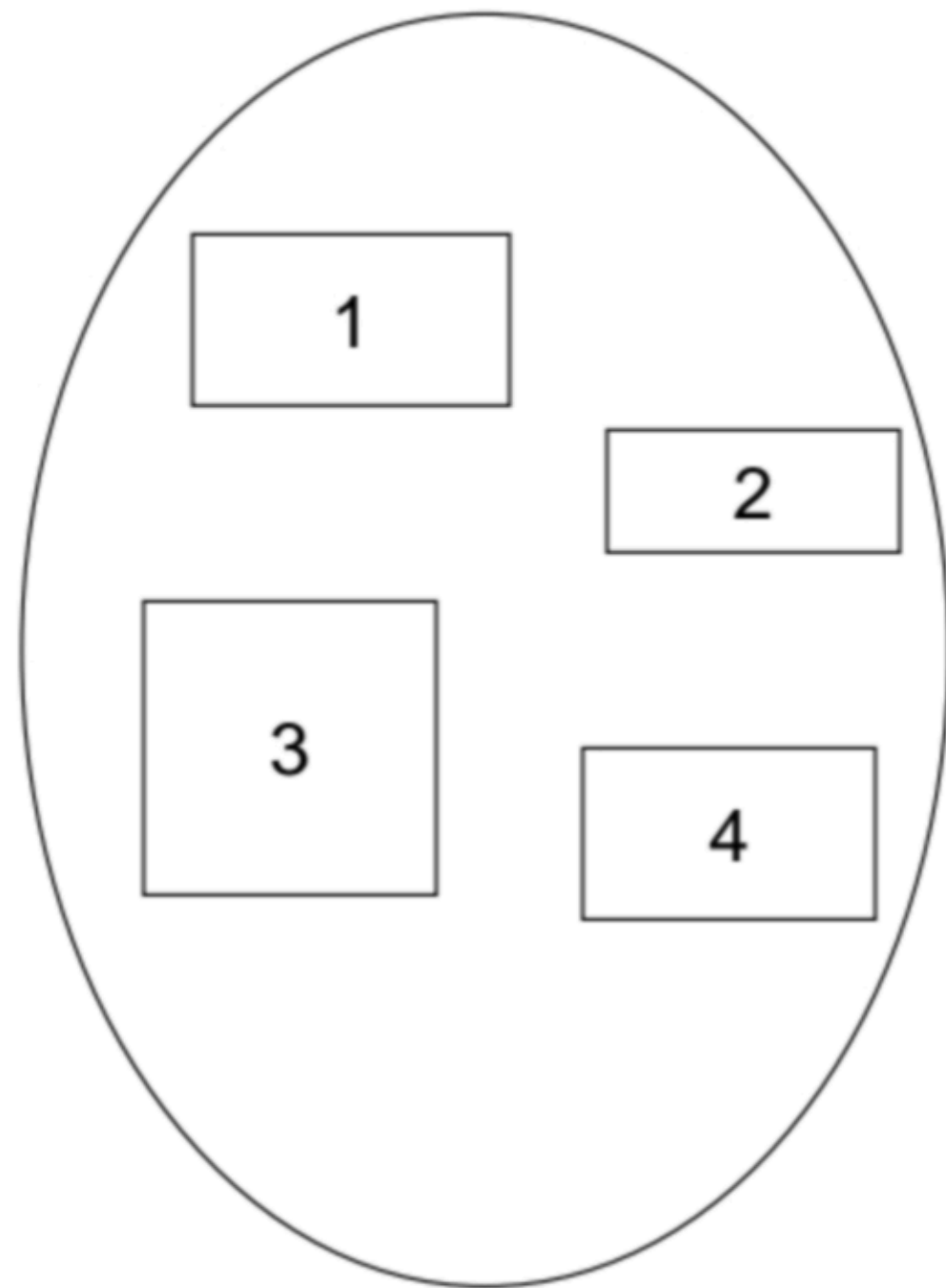
Segmentation

Logical address = <세그먼트번호, offset>

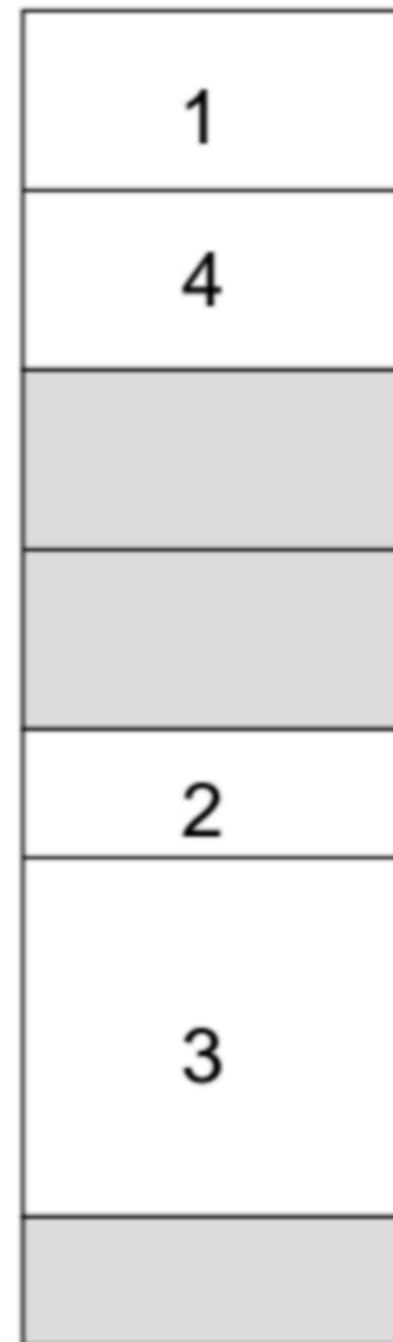


- 각각 분리된 세그먼트마다 주소에서 정해진 범위의 주소를 저장하는 테이블이 있게 됨.
- 해당 테이블이 어디있는지는 레지스터에 저장.
- 정해진 범위를 벗어나면?
→ segmentation fault

성능 및 문제점



user space



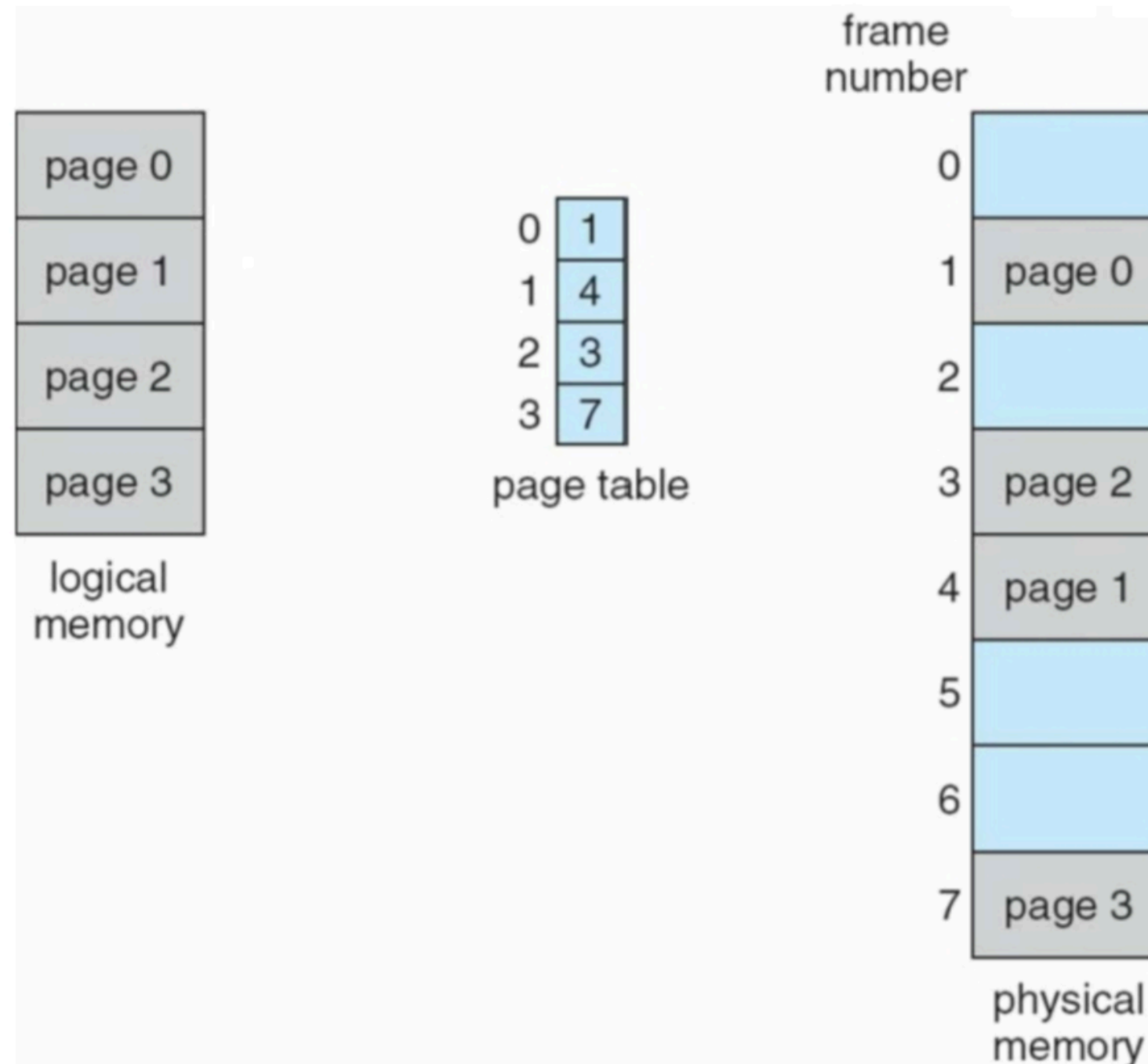
physical memory space

- 외부단편화: 연속할당 보다는 나아졌지만 여전히 존재
 - 크기를 잘 나눌수록 단편화가 적어짐.
- >최대 문제점: 나누는 존재가 개발자임
-
- 주소변환: 한 단계 추가되서 나빠짐
 - swapping도 조각난 부분을 내린다면 좋아진 편

Paging

Paging

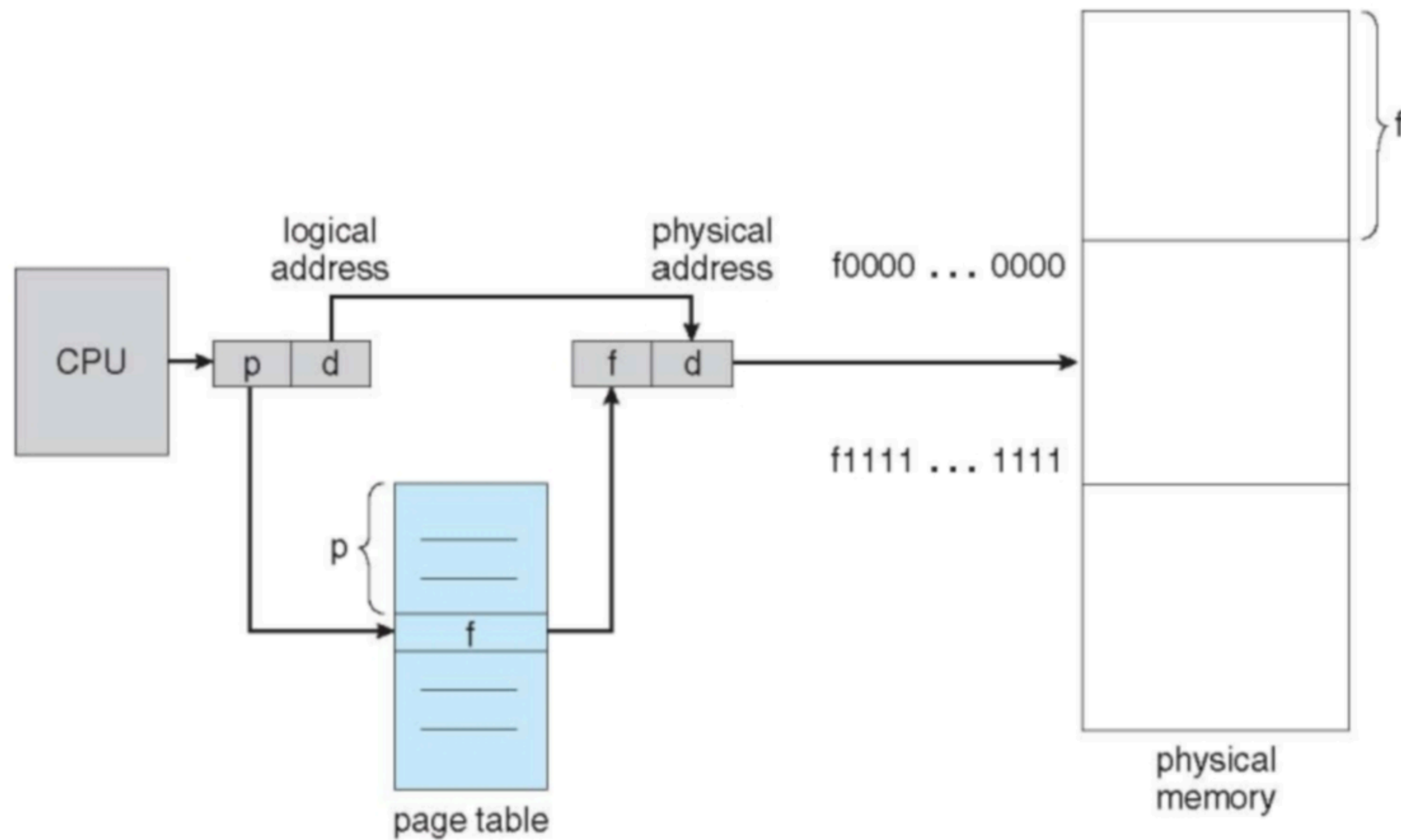
프로세스를 특정한 크기(예시:4KB)으로 잘라서 메모리에 배치
메모리도 페이지크기와 같게 잘라서 할당



- 페이지 기법도 페이지가 물리 메모리 어디에 있는지 알려주는 테이블이 있어야함.
- 마찬가지로 해당 테이블이 어디있는지는 레지스터에 저장.

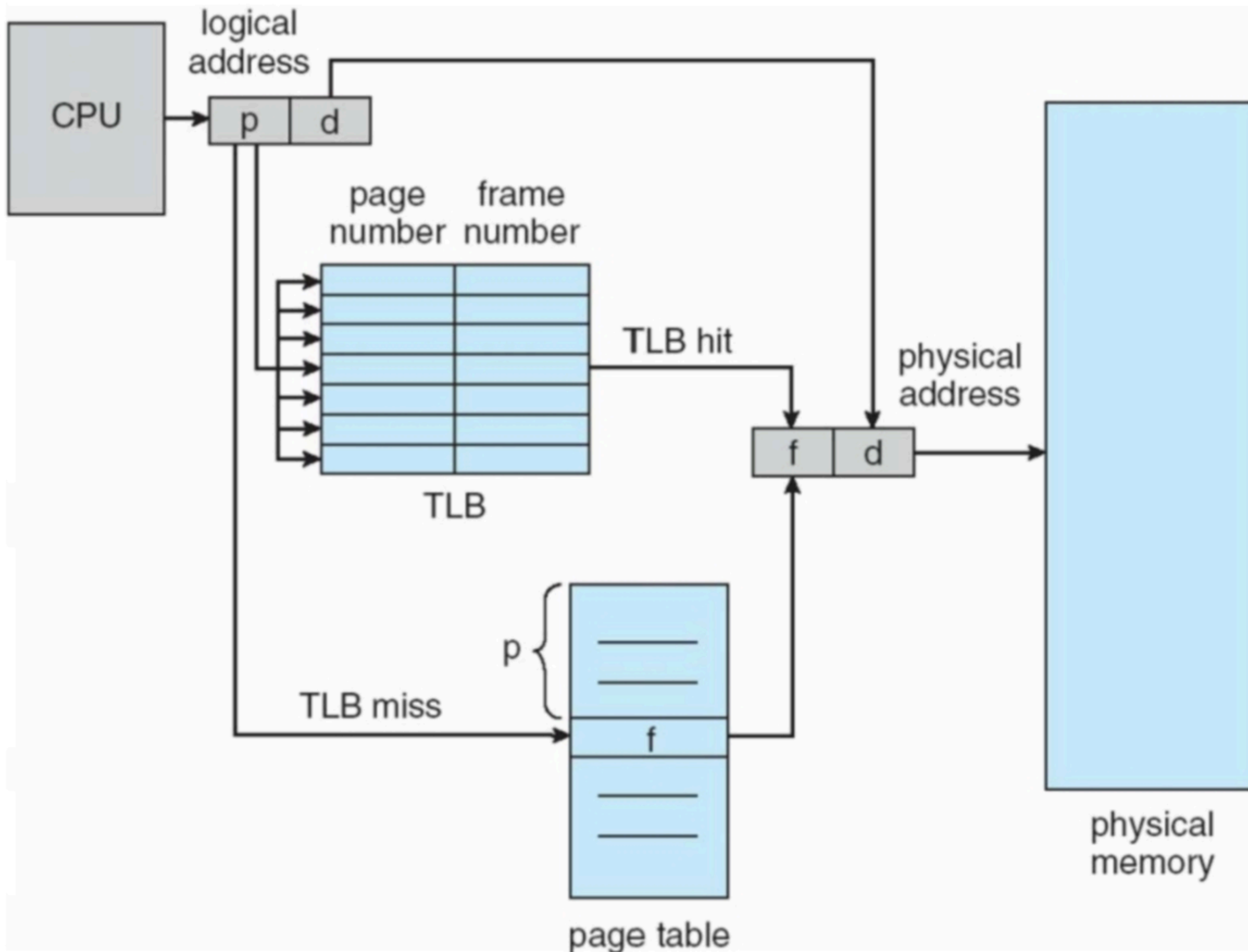
Paging

paging



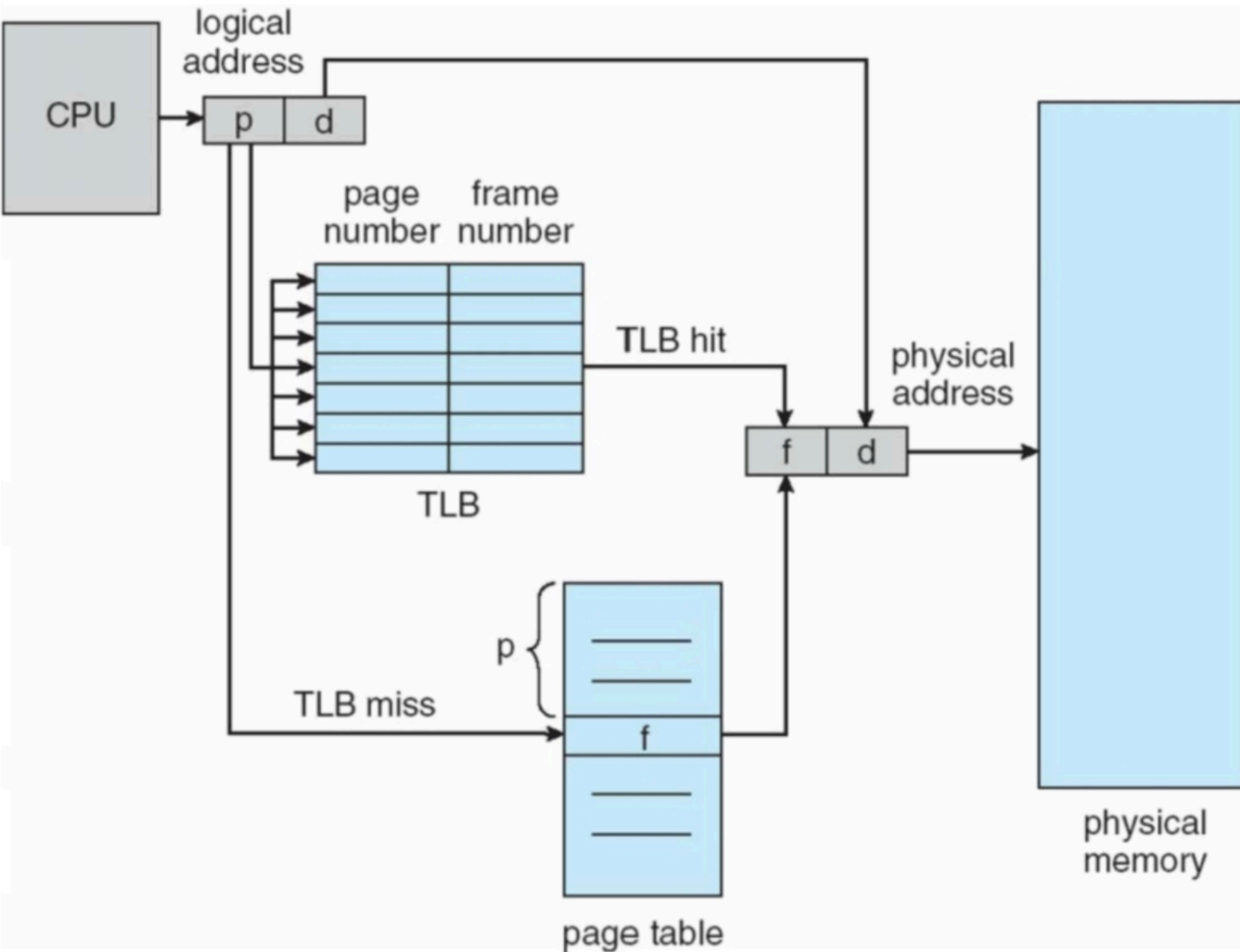
- Logical address = <페이지번호,offset>
- 테이블이 메모리에 있으므로 메모리에 2번 접근.

TLB



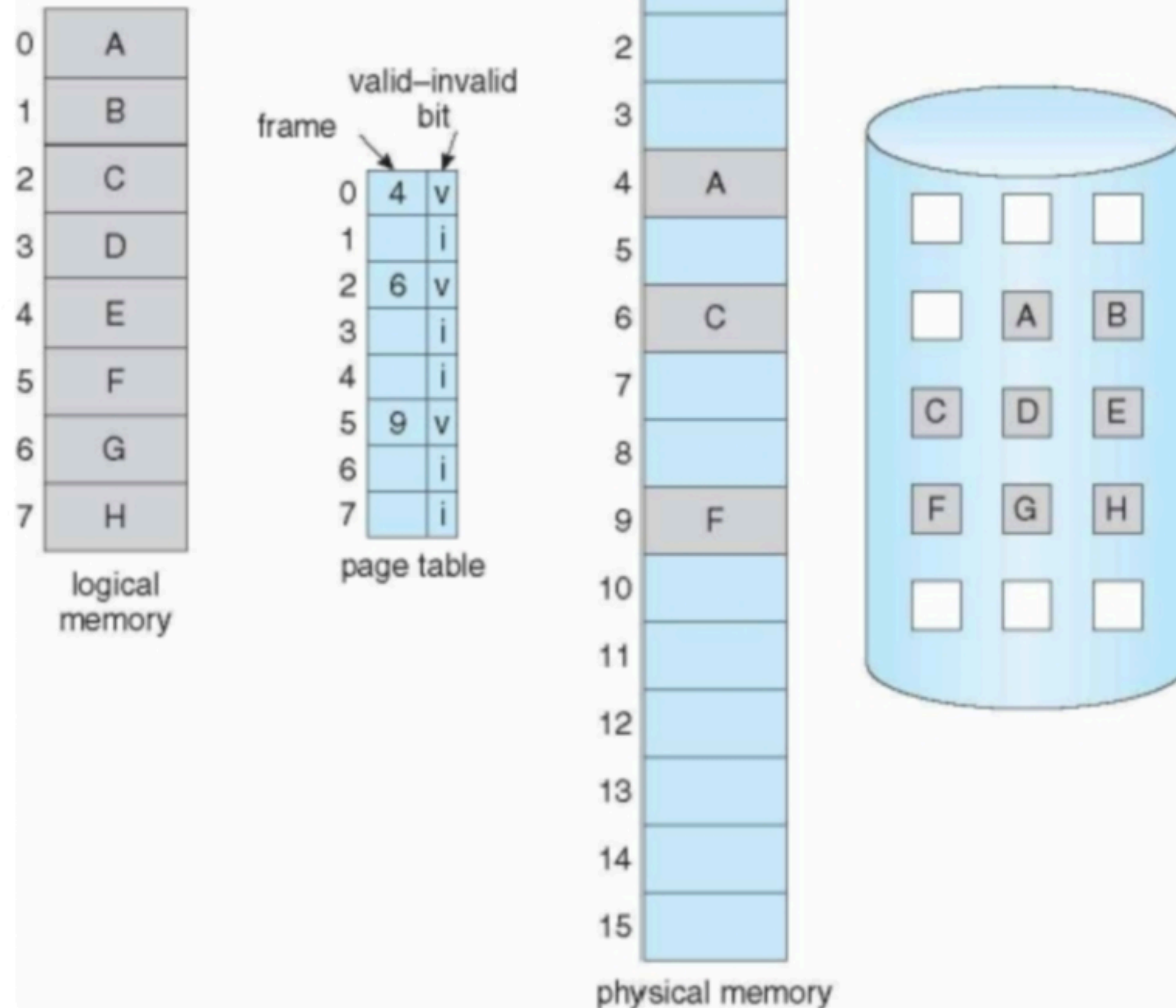
- TLB: 페이지 테이블을 캐시하기 위한 특별한 하드웨어(mmu안에 있음)
- 찾는 페이지가 있는지 병렬적으로 확인
- 매우 빠르다! 1~2ns
- TLB Hit = 1+ 100ns
- TLB MISS = 1 + 100 + 100ns

Paging 성능



- 외부단편화: 없음!
- 내부단편화: 페이지이하로 제한가능
- 주소를 2번변환? TLB는 생각보다 강력함
- 가상메모리 가능

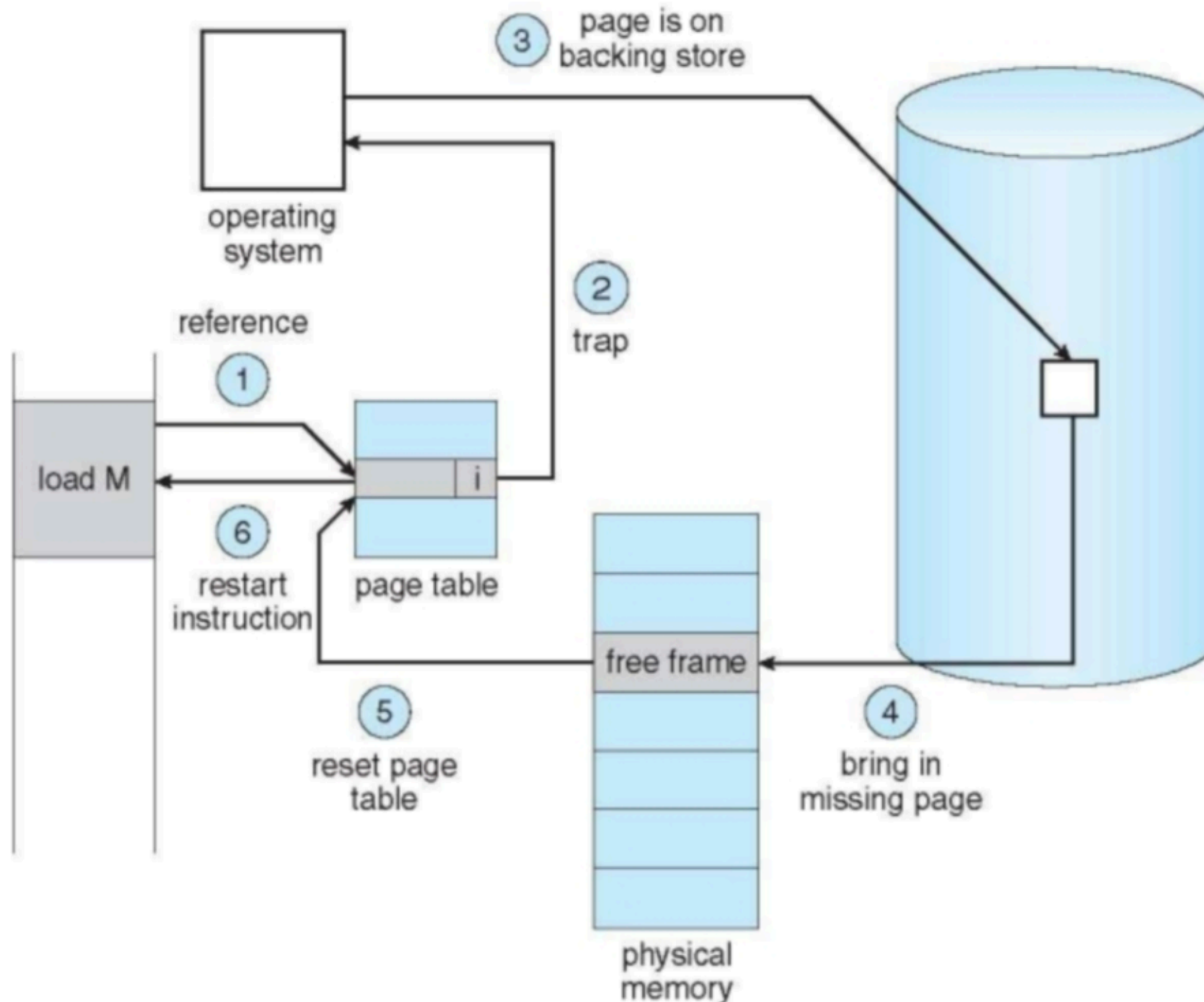
Demand paging



paging

- 프로세스를 이루는 모든 페이지가 메모리에 있을 필요가 없다!
- 초기화코드, 자주쓰는 코드..
- 페이지 테이블에 valid bit 추가
- valid bit = 현재 해당 페이지에 접근 가능한지 여부
- valid가 아니라면 storage에서 가져와야함
- 메모리에는 사용하는 페이지만 넣기
- 없는 페이지를 요청하면?
→Page Fault

Page fault



1. 페이지를 참조하는데 invalid임
2. 인터럽트 발생 (Trap)
3. 페이지 폴트 핸들러(Page Fault Handler)가 실행
4. 스토리지에서 페이지를 가져온다
5. 메모리에서 비어있는 프레임을 찾는다 혹은 페이지 교체 (Page Replacement)가 일어난다
6. 페이지가 물리 메모리에 존재하므로 실행을 이어간다.

1

페이징: 논리주소를 페이지, 물리 주소를 프레임으로 나눈다

2

단편화 문제 해결
외부 단편화 없음
내부 단편화 < 페이지 크기

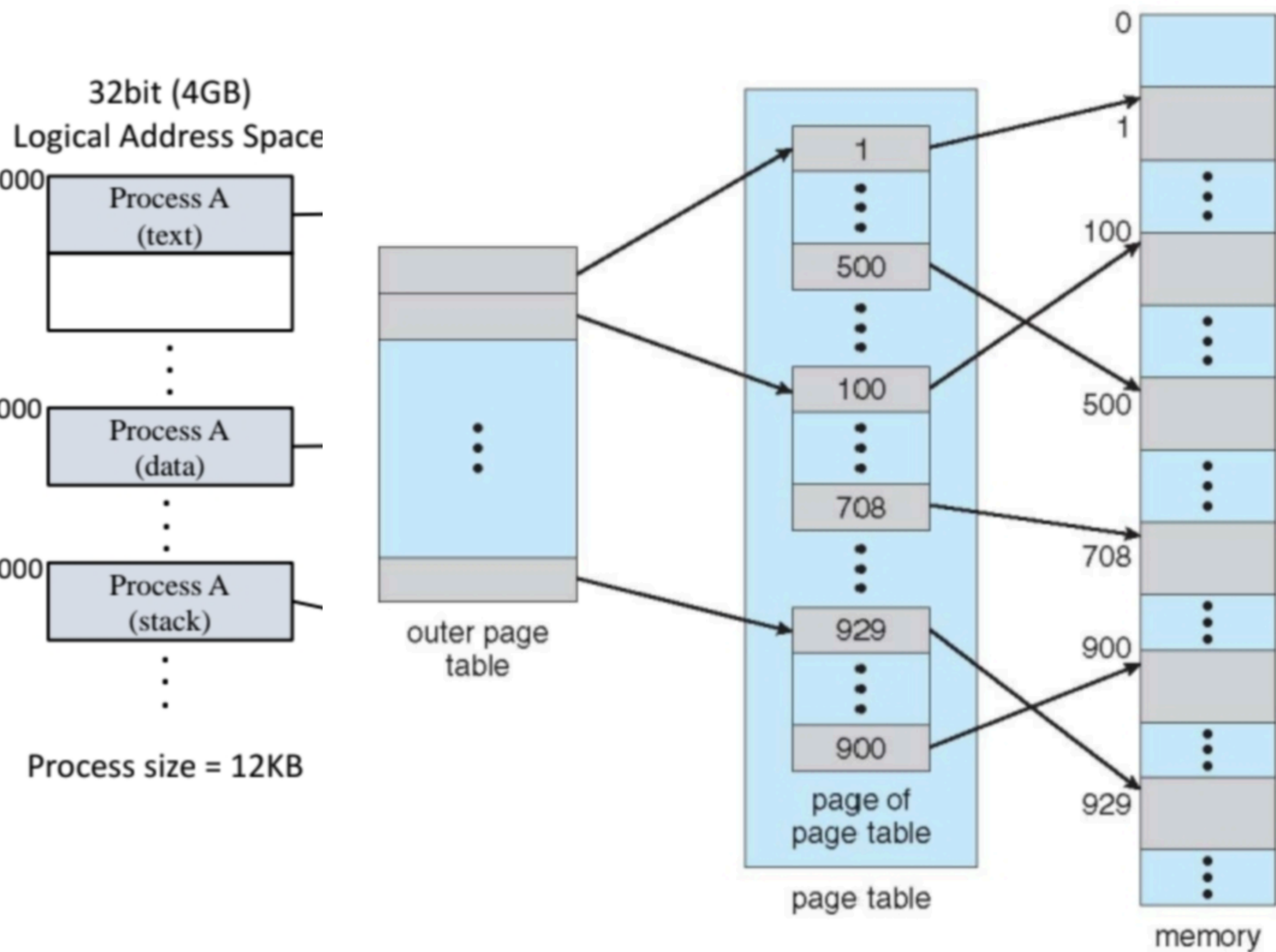
3

페이지 테이블 필요
: 메모리 2번 접근문제
TLB와 지역성 덕분에 ok

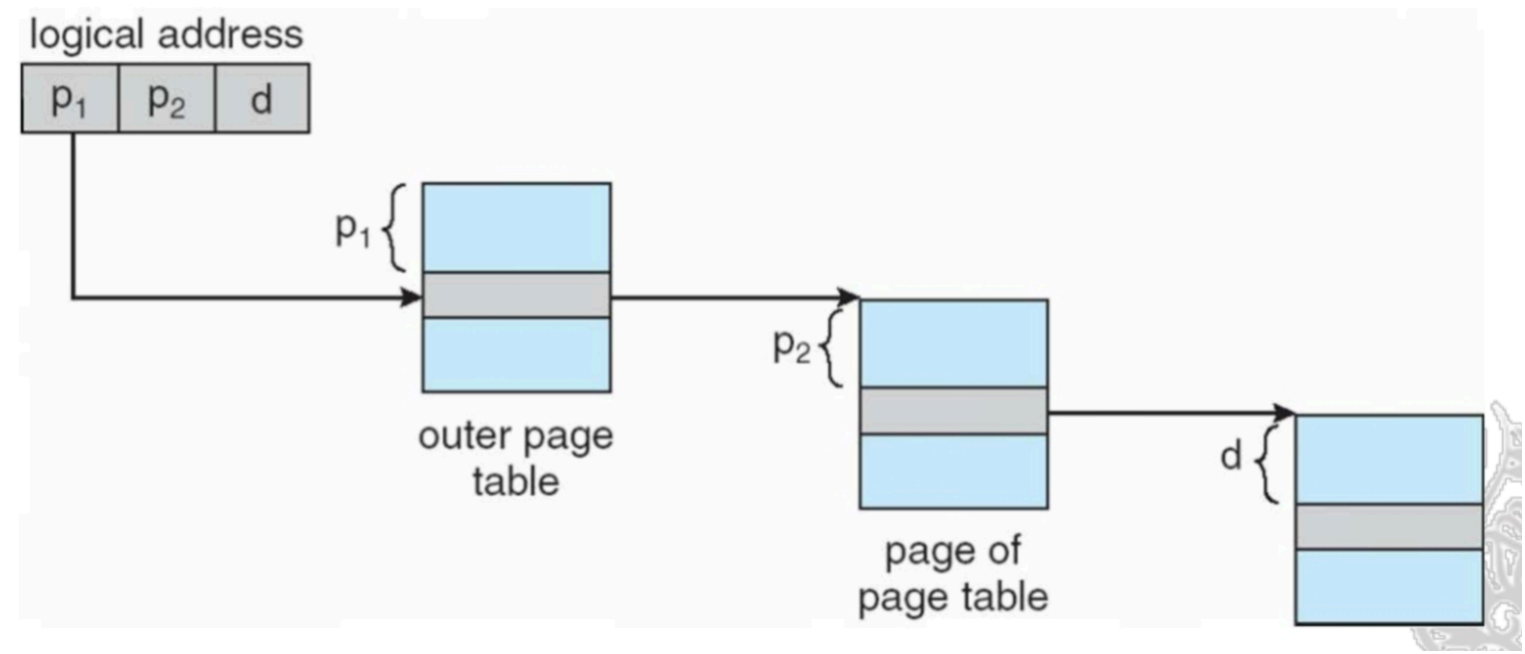
4

가상메모리(demand paging)
물리 메모리보다 큰 공간 확보
단점: **page fault**: 스토리지접근

Hierarchical page table



- 페이지 테이블 크기는? 32비트의 경우 4MB
- (4GB 공간을 4KB로 나타내기 위해.. 100만 개, 각 항목이 4byte 이므로 4MB)
- 프로세스마다 이렇게 있기는 메모리 낭비
- 페이지 테이블의 페이지 테이블을 만들자



Q&A

질의 응답

